



FICHA DE IDENTIFICACIÓN DE TRABAJO DE INVESTIGACIÓN

Título	Sistema Informático de Soporte a las Decisiones Basado en Modelos Predictivos para la Optimización de la Planificación de la Producción en la Planta Delizia Oruro	
Autor/es	Nombres y Apellidos	Código de estudiantes
	Emilio Gabriel Fernández Jiménez	88078
Fecha	10/06/2026	

Carrera	Ing. de Sistemas
Asignatura	Redes Neuronales
Grupo	A
Docente	Ing. Roger Rolando Perez Rojas
Periodo Académico	I/2026
Subsede	Oruro

Copyright © (2026) por (Emilio Gabriel Fernández Jiménez). Todos los derechos reservados.

RESUMEN:

El presente trabajo de investigación expone el desarrollo de un sistema de soporte a las decisiones diseñado bajo el paradigma de las tecnologías informáticas para la optimización logística en la planta de la empresa Delizia, ubicada en la ciudad de Oruro.

La problemática central abordada radica en la alta variabilidad de la demanda de helados, jugos y lácteos generada por los ciclos estacionales altiplánicos, lo cual introduce ineficiencias operativas crónicas debido a la falta de herramientas analíticas integradas. Para mitigar esta situación, se codificó una aplicación de escritorio modular en lenguaje Python que unifica una interfaz gráfica interactiva, control de hilos asíncronos y componentes concurrentes para evitar el congelamiento de procesos.

El núcleo del sistema incorpora una función de cálculo matemático basada en el Perceptrón Multicapa, alimentada con matrices temporales transformadas mediante ingeniería de características cíclicas basadas en funciones trigonométricas.

Los resultados prácticos del software demostraron una estabilidad operativa óptima y una alta capacidad de generalización, registrando coeficientes de determinación estadística superiores al noventa por ciento en el ajuste predictivo de las tendencias de manufactura. Asimismo, la plataforma consolidó de manera resiliente sus módulos de comunicación asíncrona mediante webhooks locales y la persistencia de datos orientada a la exportación automatizada de reportes corporativos maquetados en formato de documento portátil.

Se concluye que la implementación de esta infraestructura de software dota a la gerencia de una herramienta autónoma, técnica e independiente que reduce los costos por almacenamiento ociosos y mitiga significativamente los riesgos de desabastecimiento en el mercado regional.

Palabras clave: software, dashboard, concurrencia, regresión, persistencia, predictivo, optimización, estacionalidad, modular, inventario.

ABSTRACT:

The present research paper expounds the development of a decision support system designed under the computing technologies paradigm for logistics optimization at the Delizia enterprise plant, located in the city of Oruro.

The core problem addressed lies in the high demand variability of ice creams, juices, and dairy products generated by the highlands seasonal cycles, which introduces chronic operational inefficiencies due to the lack of integrated analytical tools. To mitigate this situation, a modular desktop application was coded in Python programming language, unifying an interactive graphical user interface, asynchronous thread control, and concurrent components to prevent process freezing.

The core of the system incorporates a mathematical calculation function based on a Multi-Layer Perceptron, fed with temporal matrices transformed through cyclical feature engineering based on trigonometric functions.

The software practical results demonstrated optimal operational stability and a high generalization capability, recording statistical determination coefficients higher than ninety percent in the predictive adjustment of manufacturing trends. Furthermore, the platform resiliently consolidated its asynchronous communication modules through local webhooks and data persistence oriented towards the automated export of corporate reports typeset in portable document format.

It is concluded that the implementation of this software infrastructure provides management with an autonomous, technical, and independent tool that reduces idle storage costs and significantly mitigates supply shortage risks in the regional market.

Key words: software, dashboard, concurrency, regression, persistence, predictive, optimization, seasonality, modular, inventory.

Tabla De Contenidos

Lista De Tablas	6
Lista De Figuras	7
Introducción	8
Capítulo 1. Planteamiento del Problema	10
1.1 Formulación del Problema	10
1.2 Objetivos	11
1.3 Justificación.....	12
1.4 Planteamiento de hipótesis	13
Capítulo 2. Marco Teórico	14
2.1 Área de Estudio/Campo de Investigación	14
2.2 Desarrollo del Marco Teórico	14
2.2.1 Tratado y Fundamentos de la Computación Neuronal.....	14
2.2.1.1 Modelos Computacionales en Estado Discreto y Continuo	14
2.2.1.2 Paradigmas de Aprendizaje: Supervisado, No Supervisado y Continuo Adaptativo.....	16
2.2.1.3 Funciones de Activación Matemáticas y su Impacto en la Convergencia Algorítmica	18
2.2.2 Arquitecturas de Redes Neuronales Biológicamente Plausibles y Sistemas Adaptativos	21
2.2.2.1 Dinámica de Redes Biológicas y Analogías para La Interpretabilidad de Cajas Negras	21
2.2.2.2 Modelos Híbridos: Integración de Redes Biológicas Simuladas con Redes Artificiales.....	23
2.2.2.3 Mecanismos de Plasticidad Biológica Aplicados al Aprendizaje Continuo en Software	25
2.2.3 Ingeniería de Software para la Simulación y Despliegue de Redes Neuronales.....	27
2.2.3.1 Patrones de Diseño e ingeniería de Software para Simulación a Gran Escala.	27

2.2.3.2 Optimización de Redes Discretas Para Sistemas Embebidos y Edge Computing	29
2.2.4 Control, Verificación y Estándares de Calidad en Software Neuronal	32
2.2.4.1 Fundamentos y Estándares de Calidad en el Desarrollo de Software Inteligente	32
2.2.4.2 Verificación Formal y Modelado de Redes Discretas Para Testing Automatizado	34
2.2.4.3 Métricas de Calidad y Evaluación de la Capacidad de Generalización Discretas vs. Continuas	36
Capítulo 3. Resultados y Discusión	39
3.1 Interfaz Gráfica de Usuario (GUI) y Diseño del Dashboard.....	39
3.2 Módulo Analítico y Procesamiento Matemático Temporal	40
3.3 Renderizado de Gráficos Dinámicos e Inteligencia Artificial Explicable (XAI).....	41
3.4 Arquitectura Concurrente mediante Hilos de Procesamiento	42
3.5 Módulo de Comunicación y Conectividad por Webhooks	43
3.6 Sistema de Persistencia y Exportación Automatizada de Reportes	45
3.7 Matriz de Síntesis Operativa y Funcional del Sistema.....	46
Capítulo 4. Conclusiones	49
Referencias	51

Lista De Tablas

Tabla 1. Tabla Comparativa de Funciones de Activación	20
Tabla 2. Matriz de Síntesis Operativa y Funcional del Sistema	47

Lista De Figuras

Figura 1. Estado Discreto y Continuo	15
Figura 2. Paradigmas de Aprendizaje	17
Figura 3. Funciones de Activación.....	19
Figura 4. Redes Biológicas y Caja Negra	22
Figura 5. Modelos Híbridos y Redes Artificiales.....	24
Figura 6. Mecanismos de Plasticidad.....	26
Figura 7. Patrones de Diseño	28
Figura 8. Redes Discretas y Sistemas Embebidos	31
Figura 9. Fundamentos y Estándares de Calidad	33
Figura 10. Verificación Formal y Modelado de Redes Discretas	35
Figura 11. Métricas de Calidad y Evaluación de la Capacidad de Generalización.....	37
Figura 12. Parámetros del Modelo Predictivo.....	40
Figura 13. Dashboard Informativo	41
Figura 14. Gráfico Explicativo.....	42
Figura 15. Aplicación en Funcionamiento	43
Figura 16. Chatbot.....	44
Figura 17. Reporte en PDF.....	46

Introducción

En el dinámico entorno industrial contemporáneo, la optimización de los procesos de distribución y la planificación de la producción representan pilares fundamentales para garantizar la competitividad y sostenibilidad de las empresas de consumo masivo.

Dentro del contexto boliviano, la planta de la empresa Delizia, ubicada en la ciudad de Oruro, se enfrenta al constante desafío de gestionar de manera eficiente la demanda de sus principales líneas de productos, las cuales abarcan helados, jugos y derivados lácteos. Las marcadas fluctuaciones climáticas y las particularidades estacionales de la región altiplánica generan un comportamiento altamente variable en el consumo, lo que dificulta la toma de decisiones operativas si se depende únicamente de metodologías tradicionales o estimaciones empíricas.

Ante este escenario, surge la necesidad apremiante de diseñar e implementar soluciones informáticas avanzadas que actúen como sistemas de soporte a las decisiones, capaces de procesar volúmenes de datos históricos y transformarlos en proyecciones de mercado altamente fiables.

El presente trabajo de investigación se orienta al desarrollo de una aplicación de escritorio modular e integral, concebida bajo el paradigma de las tecnologías informáticas aplicadas a la gestión empresarial.

La solución desarrollada no solo se limita a la interfaz de usuario, sino que unifica de forma sinérgica múltiples componentes tecnológicos de última generación, incluyendo la programación concurrente para evitar el congelamiento de procesos en entornos de alta carga computacional, la integración de servicios de comunicación asíncrona mediante protocolos HTTP y la automatización de reportes corporativos en formatos de alta fidelidad.

Al empaquetar estas herramientas en un entorno gráfico intuitivo y dinámico, se provee a la gerencia de la planta de una plataforma informática robusta que agiliza la visualización de escenarios futuros y mitiga los riesgos asociados tanto al desabastecimiento de inventarios como a la sobreproducción de mercancías.

Es fundamental destacar que, para sustentar la robustez algorítmica de las funciones de cálculo embebidas en el software, este documento realiza una reconstrucción teórica total y exhaustiva de los fundamentos que rigen los modelos computacionales avanzados.

Más allá de la implementación práctica en la interfaz informática, el estudio desglosa meticulosamente la evolución, las arquitecturas y los mecanismos de aprendizaje de las redes neuronales artificiales, analizándolas desde una perspectiva matemática y lógica independiente.

Esta profunda base conceptual es la que, valida la incorporación del Perceptrón Multicapa como el motor analítico del sistema, demostrando cómo los principios de la computación inspirada en la biología pueden ser asimilados por el desarrollo de software moderno para resolver problemas complejos de predicción estacional y optimización en la cadena de suministro.

Capítulo 1. Planteamiento del Problema

1.1 Formulación del Problema

La gestión operativa y la planificación de la producción en la planta de la empresa Delizia, ubicada en la ciudad de Oruro, se encuentran condicionadas por un entorno de alta variabilidad en el consumo de sus líneas de productos principales, que incluyen helados, jugos y derivados lácteos. Las particulares condiciones climáticas de la región altiplánica y las fluctuaciones estacionales introducen comportamientos dinámicos y no lineales en la demanda del mercado, lo que dificulta que la gerencia pueda realizar proyecciones precisas mediante métodos de estimación tradicionales o herramientas ofimáticas convencionales.

Esta carencia de un soporte analítico automatizado e integrado genera desajustes continuos en la cadena de suministro, derivando ya sea en escenarios de sobreproducción que elevan los costos de almacenamiento y mermas de productos perecederos, o en situaciones de desabastecimiento que impactan negativamente en los niveles de servicio y la satisfacción del cliente.

El núcleo del problema radica en que los datos históricos generados por la actividad comercial de la planta no son explotados a través de una infraestructura de software dedicada que centralice, procese y exponga la información de manera clara, interactiva y oportuna para los tomadores de decisiones.

La ausencia de herramientas informáticas concurrentes que ejecuten tareas analíticas sin interrumpir el entorno de usuario, la falta de módulos que automaticen la generación de reportes ejecutivos en formatos portables y la inexistencia de canales de consulta interactivos limitan la capacidad de respuesta táctica de la empresa ante las variaciones del mercado regional.

Por lo tanto, la interrogante principal que guía la presente investigación se define de la siguiente manera: ¿En qué medida el desarrollo e implementación de una aplicación informática de escritorio orientada al soporte de decisiones, basada en procesamiento concurrente y equipada

con un módulo algorítmico de proyección estacional, permitirá optimizar el proceso de planificación de la producción en la planta Delizia de la ciudad de Oruro?

1.2 Objetivos

Desarrollar una aplicación informática de escritorio basada en arquitectura modular y procesamiento concurrente para actuar como un sistema de soporte a las decisiones en la planta Delizia de Oruro, integrando un módulo algorítmico de proyección estacional y un componente automatizado de reportes corporativos para optimizar la planificación en la producción de helados, jugos y lácteos.

- Diseñar e implementar una interfaz gráfica de usuario interactiva y fluida que permita la visualización dinámica de indicadores clave de rendimiento comercial y tendencias de mercado para cada línea de producto.
- Desarrollar un entorno de ejecución síncrono y asíncrono mediante el uso de hilos secundarios de procesamiento, garantizando que las operaciones de cálculo matemático intensivo y la carga de datos no comprometan la estabilidad ni la fluidez de la interfaz gráfica principal.
- Codificar un módulo analítico que asimile matrices de datos históricos transformadas mediante ingeniería de características cíclicas y temporales, sirviendo como la función de cálculo matemático embebida para generar proyecciones futuras fiables.
- Integrar un servicio de comunicación informática asíncrono que permita la conectividad con un servidor local a través de protocolos de transferencia de hipertexto y webhooks, implementando mecanismos de contingencia para la interacción interactiva en texto.

- Implementar un componente de persistencia de datos y salida de información capaz de compilar automáticamente los resultados del análisis predictivo para estructurar y exportar reportes corporativos nativos en formato PDF con diseño editorial e independiente.

1.3 Justificación

La presente investigación se justifica de manera integral por la necesidad de dotar a la planta Delizia de Oruro de una infraestructura tecnológica avanzada que reemplace la dependencia de estimaciones empíricas por un entorno automatizado y riguroso de soporte a las decisiones.

Desde la perspectiva de la ingeniería de software y las tecnologías informáticas, el proyecto demuestra la viabilidad técnica de unificar en una sola herramienta de escritorio el procesamiento asíncrono y concurrente mediante subprocesos, la manipulación de arreglos de datos estructurados, el renderizado de gráficos analíticos interactivos y la persistencia de información en documentos portables nativos.

Esta solución resuelve de forma directa el problema clásico de congelamiento de interfaces durante cálculos de alta carga de cómputo, validando el uso de librerías de código abierto para construir aplicaciones empresariales estables y modulares.

Al automatizar estos flujos y proveer proyecciones matemáticas altamente ajustadas a los ciclos climáticos y de mercado locales, la aplicación genera un impacto directo en la salud financiera de la empresa, reduciendo costos por sobreproducción o almacenamiento ocioso, y evitando quiebres de existencias en periodos de alta demanda.

De este modo, la optimización informática se traduce en una planificación logística equilibrada que beneficia tanto a la eficiencia corporativa como a la regularidad operativa de su personal y la estabilidad en el abastecimiento de productos alimenticios esenciales para la comunidad orureña.

1.4 Planteamiento de hipótesis

El desarrollo de una aplicación informática de escritorio modular, que unifique una interfaz gráfica interactiva, procesamiento asíncrono concurrente y un módulo analítico basado en el modelado matemático de tendencias y estacionalidades semanales, proporcionará a la gerencia de la planta Delizia Oruro proyecciones de demanda estables con un coeficiente de determinación estadístico superior al umbral mínimo aceptable de fiabilidad, permitiendo una toma de decisiones informada que reduzca los márgenes de error en la planificación de la producción de helados, jugos y derivados lácteos..

Capítulo 2. Marco Teórico

2.1 Área de Estudio/Campo de Investigación

Área de estudio: Ingeniería de Sistemas – Redes Neuronales.

2.2 Desarrollo del Marco Teórico

2.2.1 Tratado y Fundamentos de la Computación Neuronal

2.2.1.1 Modelos Computacionales en Estado Discreto y Continuo

La computación neuronal moderna fundamenta su capacidad operativa en la abstracción matemática de los procesos biológicos de procesamiento de información, clasificando sus arquitecturas según la naturaleza de sus estados en discretos o continuos.

Los modelos computacionales en estado discreto operan sobre dominios de tiempo y espacio finitos y cuantizados, donde la transferencia de información entre nodos se realiza mediante pulsos, transiciones binarias o pasos lógicos bien definidos.

Este enfoque es sumamente valorado en la ingeniería de software y el diseño de sistemas de información debido a su predictibilidad algorítmica, su facilidad para la verificación formal y su acoplamiento directo con la arquitectura binaria de los procesadores convencionales, lo que optimiza el uso de memoria y los ciclos de reloj durante fases de cálculo masivo.

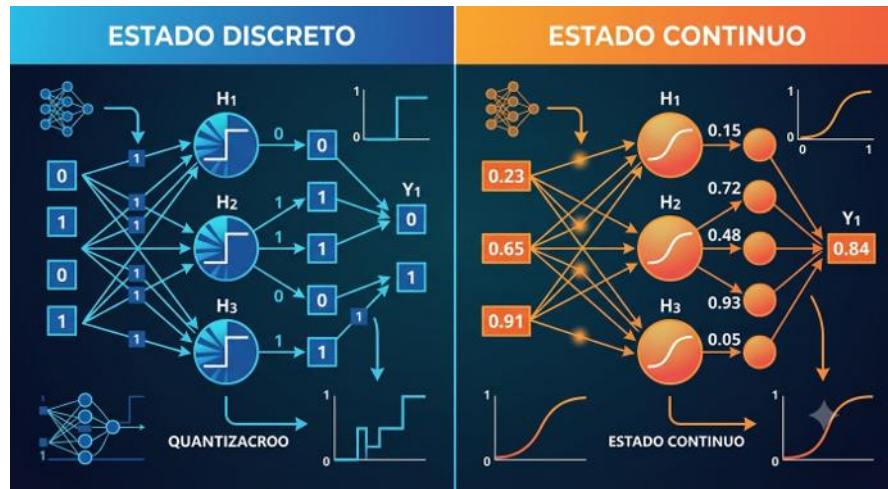
Por otro lado, los modelos en estado continuo modelan la activación y la propagación de datos mediante funciones matemáticas continuas y derivables en el tiempo, simulando una tasa de disparo neuronal que varía de forma suavizada.

Estas arquitecturas son capaces de representar dinámicas sumamente complejas, comportamientos caóticos y variaciones no lineales sutiles dentro de un flujo de datos, lo que las hace ideales para el procesamiento de señales analógicas en tiempo real.

Sin embargo, su implementación en sistemas digitales de escritorio requiere procesos de aproximación numérica y discretización temporal, lo que incrementa la carga computacional del

procesador e introduce un margen de error por redondeo que debe ser controlado algorítmicamente.

Figura 1. Estado Discreto y Continuo



Fuente: *Elaboración Propia*

En el contexto del desarrollo de software analítico para el soporte a las decisiones, la elección entre un paradigma discreto o continuo define la estructura de las estructuras de datos y las matrices de cómputo.

Mientras que los estados continuos permiten calcular aproximaciones de regresión extremadamente suaves a partir de entradas normalizadas, los estados discretos se imponen en las rutinas de control y en la segmentación secuencial de los procesos de entrenamiento.

La asimilación de ambos enfoques dentro de un sistema informático homogéneo permite que las capas lógicas operen bajo un esquema de discretización temporal por días o semanas, mientras que las capas de cálculo matemático interno resuelven las ecuaciones de optimización sobre un espacio de pesos continuos y derivables.

La confluencia de estos modelos da origen a sistemas híbridos estables donde la estabilidad de la aplicación depende de la correcta sincronización de ambos estados.

La ingeniería de software moderna aprovecha la rigidez estructural de los modelos discretos para construir las interfaces de usuario, la persistencia documental y el control de flujos asíncronos, delegando a las funciones continuas únicamente el ajuste fino de los coeficientes de predicción.

Esta separación funcional garantiza que el software mantenga un rendimiento óptimo, permitiendo que un algoritmo de cómputo avanzado sea encapsulado de forma segura dentro de una aplicación corporativa estándar sin comprometer los recursos de hardware del sistema anfitrión.

2.2.1.2 Paradigmas de Aprendizaje: Supervisado, No Supervisado y Continuo Adaptativo

Los mecanismos de adaptación algorítmica en la computación neuronal se estructuran a través de paradigmas de aprendizaje bien definidos, los cuales determinan cómo el sistema altera sus coeficientes internos para asimilar nuevos patrones de datos.

El aprendizaje supervisado constituye el pilar fundamental para las tareas de regresión y proyección analítica, caracterizándose por requerir una matriz de entrenamiento compuesta explícitamente por pares de entradas y salidas deseadas, denominadas etiquetas de referencia. Durante el proceso de optimización, el software computa la discrepancia entre el valor generado por la red y el valor real del historial comercial, utilizando esta señal de error para retropropagar los ajustes matemáticos y minimizar de forma iterativa las pérdidas del sistema.

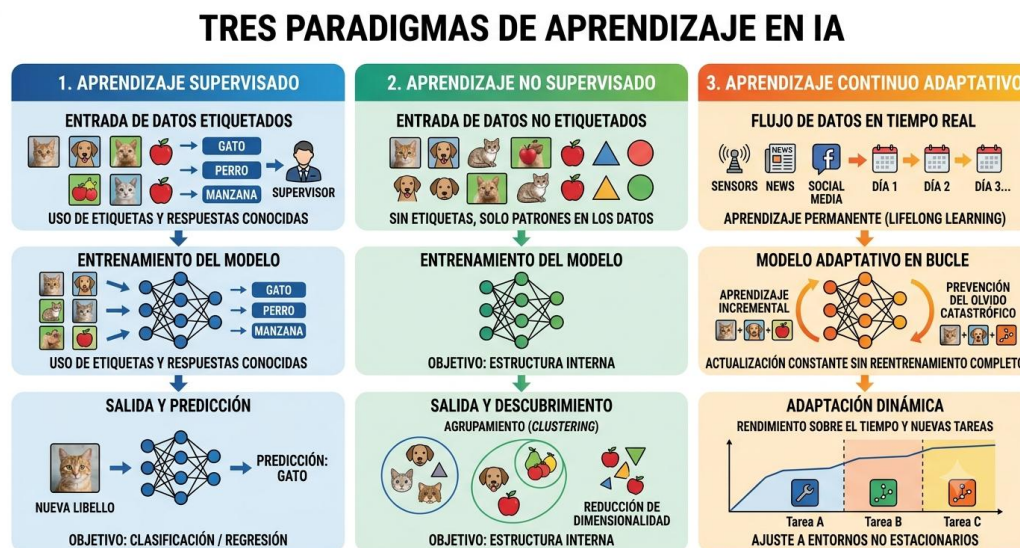
En contraste, el aprendizaje no supervisado opera sobre estructuras de datos que carecen de etiquetas u objetivos predefinidos, obligando al algoritmo a descubrir por sí mismo las correlaciones ocultas, las agrupaciones naturales y las regularidades estadísticas intrínsecas del conjunto de datos.

Este paradigma es ampliamente utilizado en las fases previas de minería de datos y segmentación de mercados, ya que permite identificar perfiles de comportamiento complejos sin la intervención o el sesgo de un operador humano. En las aplicaciones de soporte a las decisiones, el enfoque no supervisado puede actuar como un módulo de filtrado de anomalías, aislando

registros corruptos o ruidos atípicos antes de que los vectores de información sean transferidos a los motores de cálculo principal.

Por su parte, el paradigma de aprendizaje continuo adaptativo representa la evolución hacia sistemas de software resilientes capaces de actualizar sus estructuras de conocimiento en tiempo real y de forma interactiva sin sufrir del fenómeno de olvido catastrófico. Este enfoque se inspira directamente en los procesos de plasticidad biológica, permitiendo que la aplicación asimile nuevas tendencias de mercado e incorpore variables emergentes sobre la marcha, sin necesidad de detener los servicios corporativos para ejecutar un reentrenamiento completo desde cero.

Figura 2. Paradigmas de Aprendizaje



Fuente: *Elaboración Propia*

La implementación informática de este paradigma exige la creación de estructuras de control dinámicas que protejan los pesos sinápticos consolidados mientras se abren canales de flexibilidad matemática para los nuevos flujos de información.

Para un sistema informático enfocado en la cadena de suministro industrial, la comprensión e integración de estos paradigmas resulta crítica al momento de programar las reglas de negocio y los flujos de control de la aplicación.

Un software de escritorio robusto debe ser diseñado para canalizar las matrices históricas a través de una función de aprendizaje supervisado para fijar la línea base de las proyecciones, mientras implementa rutinas continuas adaptativas para ajustar los coeficientes ante volatilidades imprevistas introducidas por el usuario.

Esta combinación paradigmática asegura que la plataforma mantenga su relevancia analítica a lo largo del tiempo, transformando el código estático en una herramienta dinámica de soporte táctico corporativo.

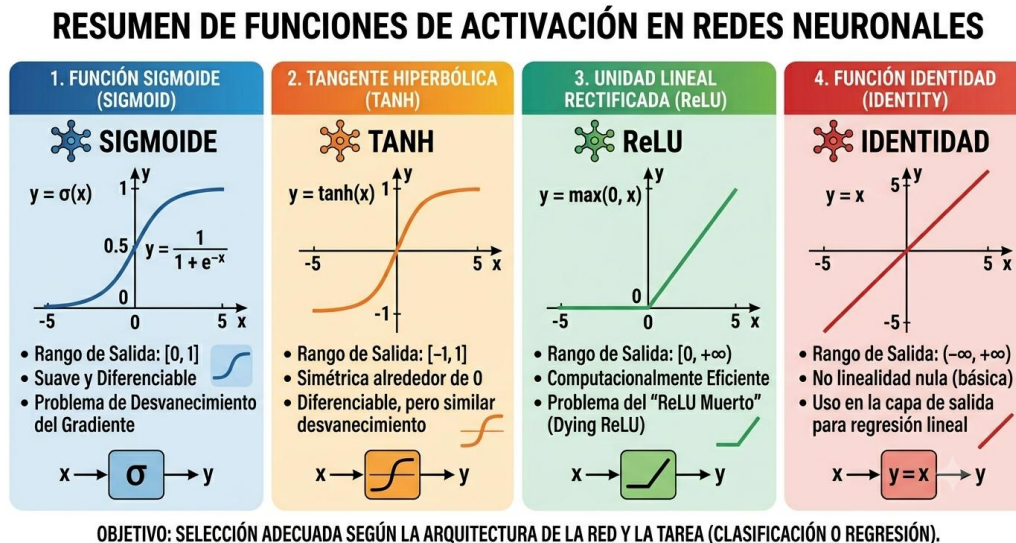
2.2.1.3 Funciones de Activación Matemáticas y su Impacto en la Convergencia Algorítmica

Las funciones de activación representan el componente matemático encargado de introducir la no linealidad dentro de los modelos de computación neuronal, permitiendo a los algoritmos romper la barrera de las transformaciones lineales simples y aproximar superficies de datos de alta complejidad. Estas funciones operan como un umbral de transferencia o filtro de energía que transforma el valor neto de entrada recibido por un nodo en una señal de salida acotada o rectificada.

La selección de la función de activación dentro de las capas ocultas y de salida de una red neuronal define de forma directa la velocidad de convergencia, el consumo de recursos de procesamiento y la estabilidad matemática de los gradientes durante la ejecución de los algoritmos de entrenamiento.

Históricamente, las funciones de tipo sigmoideal y la tangente hiperbólica dominaron el campo del modelado predictivo debido a su capacidad para comprimir el espacio numérico infinito en rangos estrictamente acotados entre cero y uno, o menos uno y uno, respectivamente.

Figura 3. Funciones de Activación



Fuente: *Elaboración Propia*

No obstante, desde la perspectiva de la ingeniería de software moderno, estas funciones introducen complicaciones técnicas severas conocidas como el problema del desvanecimiento del gradiente, donde los valores de las derivadas se aproximan a cero en los extremos de la curva, deteniendo por completo el aprendizaje de las capas iniciales del sistema.

Para contrarrestar esta ineficiencia computacional, se imponen funciones de tipo lineal rectificada, las cuales mantienen una derivada constante para valores positivos, acelerando significativamente el procesamiento matemático y reduciendo el uso de operaciones de punto flotante en el procesador.

El impacto de estas funciones en la convergencia algorítmica es un factor crítico al programar sistemas analíticos estables, ya que una mala elección puede derivar en oscilaciones caóticas, saturación de memoria o un estancamiento prematuro en mínimos locales subóptimos.

Al diseñar la lógica de cálculo embebida en un software de soporte a las decisiones, los desarrolladores deben evaluar el comportamiento de las funciones de activación frente a la

naturaleza de los vectores de entrada escalados, garantizando que el optimizador matemático pueda ajustar los pesos de manera uniforme a lo largo de las iteraciones programadas.

La correcta calibración de estos componentes asegura que el Coeficiente de Determinación alcance niveles de fiabilidad óptimos de forma rápida y reproducible.

Para sintetizar con precisión las propiedades, ventajas de desarrollo y el impacto directo que poseen las principales funciones de activación utilizadas en la ingeniería de software inteligente, se presenta a continuación una matriz analítica comparativa.

Esta tabla evalúa los componentes matemáticos desde una perspectiva puramente informática de rendimiento y convergencia de código.

Tabla 1. Tabla Comparativa de Funciones de Activación

Función de Activación	Formulación Matemática/ Comportamiento	Coste Computacional en Procesamiento	Comportamiento del Gradiente y Convergencia	Impacto Operativo en el Software Predictivo
Sigmoide (Logistic)	Comprime las salidas de forma asintótica en el rango estricto de $[0, 1]$.	Alto, requiere el cálculo de funciones exponenciales continuas.	Tiende al desvanecimiento del gradiente en valores extremos, ralentizando el entrenamiento.	Ideal para capas de salida que modelan probabilidades o clasificaciones discretas.
Tangente Hiperbólica (Tanh)	Centrada en cero, comprime los vectores en el rango de $[-1, 1]$.	Alto, exige operaciones algebraicas de punto flotante complejas.	Mayor fluidez que la sigmoide, pero mantiene el riesgo de saturación en redes profundas.	Utilizada en el procesamiento de series temporales con fluctuaciones negativas y positivas.

Lineal Rectificada (ReLU)	Retorna cero para entradas negativas y mantiene una identidad lineal $f(x) = x$ para positivas.	Extremadamente bajo, se resuelve mediante una operación lógica de comparación simple.	Gradiente constante de 1.0 para valores positivos, eliminando el desvanecimiento y acelerando la convergencia.	Es la opción estándar para capas ocultas concurrentes; optimiza el rendimiento y evita el lag de la interfaz.
Identidad (Linear)	Mantiene el valor de entrada sin ninguna transformación o límite: $f(x) = x$.	Nulo, no añade ninguna operación de procesamiento adicional.	Constante en todo el dominio numérico, conserva la proporcionalidad lineal de los datos.	Imprescindible en la capa de salida de módulos de regresión para proyectar volúmenes continuos de producción.

Fuente: *Elaboración Propia*

2.2.2 Arquitecturas de Redes Neuronales Biológicamente Plausibles y Sistemas Adaptativos

2.2.2.1 Dinámica de Redes Biológicas y Analogías para La Interpretabilidad de Cajas Negras

El auge de los sistemas inteligentes dentro de las tecnologías de la información ha puesto de manifiesto una limitación crítica conocida como el problema de la caja negra, donde las arquitecturas avanzadas procesan datos y generan proyecciones precisas, pero carecen de transparencia en sus capas lógicas internas.

Para solucionar este desafío, la ingeniería de software recurre al estudio de la dinámica de las redes biológicas, estableciendo analogías estructurales que permiten mapear el flujo de señales.

Al analizar cómo el cerebro biológico distribuye estímulos a través de circuitos interconectados y módulos especializados, los desarrolladores de software pueden diseñar herramientas de interpretabilidad que traducen matrices de pesos numéricos abstractos en explicaciones funcionales legibles para el usuario humano.

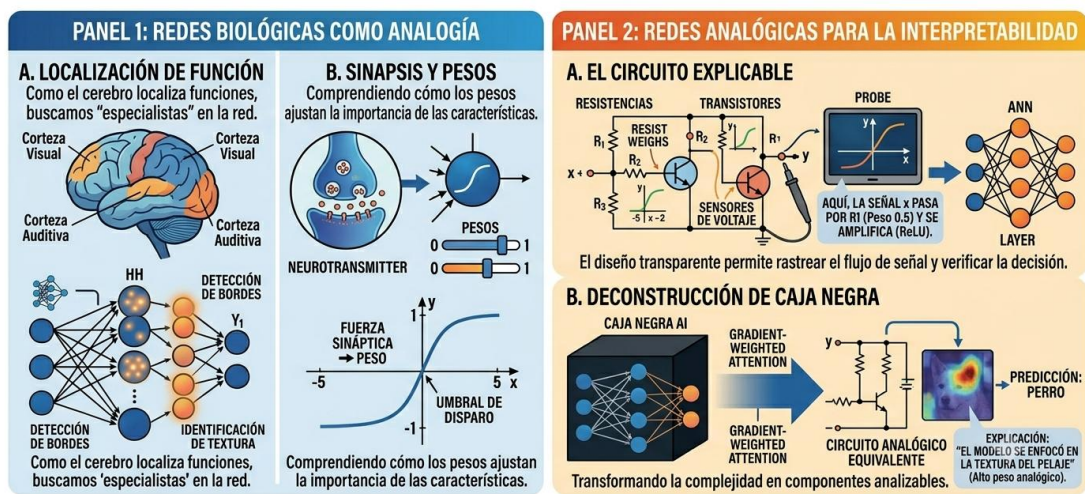
La analogía principal radica en la equivalencia entre la sinapsis biológica y los coeficientes de ponderación en un entorno de software, donde la fuerza de un enlace determina la relevancia de una variable de entrada dentro del sistema analítico.

En una red biológica, el paso de información está condicionado por la acumulación de potenciales de acción, un fenómeno que en la computación digital se emula mediante la agregación de vectores lógicos y el filtrado por funciones de activación.

Al rastrear qué rutas lógicas de la aplicación de escritorio registran la mayor tasa de activación ante fluctuaciones de datos, se logra identificar qué factores estacionales o de mercado están guiando de manera prioritaria la toma de decisiones algorítmica.

Figura 4. Redes Biológicas y Caja Negra

DESMITIFICANDO LA CAJA NEGRA: CAMINOS HACIA LA INTERPRETABILIDAD



OBJETIVO: UTILIZAR ANALOGÍAS Y SISTEMAS EXPLICABLES PARA VERIFICAR, AUDITAR Y CONFIAR EN LAS DECISIONES DE IA COMPLEJAS.

Fuente: *Elaboración Propia*

Esta correspondencia conceptual proporciona las bases teóricas para la construcción de módulos de Inteligencia Artificial Explicable (XAI) dentro de las aplicaciones corporativas. Al mapear las curvas de ajuste predictivo y conectarlas con escuchadores de eventos interactivos, el software simula el comportamiento de un observador de dinámicas neuronales, decodificando los estados internos del modelo matemático para justificar picos o valles de producción.

De este modo, la complejidad abstracta de los arreglos multidimensionales se disuelve en una lógica de negocio comprensible, fortaleciendo la confianza de los operadores de la planta en las salidas provistas por el sistema de información.

Finalmente, la adopción de estas analogías biológicas en el diseño de software optimiza los procesos de depuración y auditoría algorítmica.

Comprender que una red neuronal artificial funciona como un tejido dinámico interconectado permite a los ingenieros de sistemas anticipar comportamientos anómalos, saturación de canales o sesgos informáticos analizando la topología del modelo.

Esta perspectiva transforma la programación de código predictivo de un simple ejercicio estadístico a una construcción de software robusta, transparente y alineada con las necesidades operativas de la auditoría de sistemas y el control de calidad tecnológico.

2.2.2.2 Modelos Híbridos: Integración de Redes Biológicas Simuladas con Redes Artificiales

La evolución de la computación inspirada en la biología ha impulsado el desarrollo de modelos híbridos, los cuales consisten en sistemas informáticos que unifican la precisión matemática de las redes neuronales artificiales con la resiliencia y el procesamiento temporal propio de las redes biológicas simuladas.

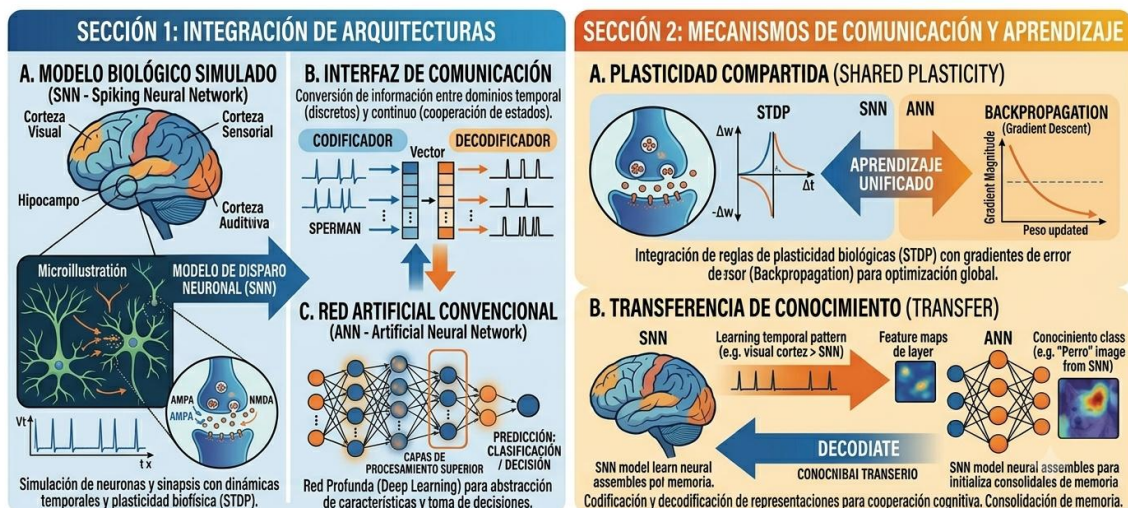
Mientras que las arquitecturas artificiales tradicionales sobresalen en el ajuste de curvas no lineales y la regresión estadística sobre matrices estáticas, la simulación de redes biológicas aporta capacidades avanzadas para el manejo de la asincronía y el ordenamiento cronológico de eventos.

Esta integración se consolida en el desarrollo de software a través de interfaces lógicas que permiten a ambos paradigmas intercambiar matrices de características para refinar las salidas del sistema.

En el diseño de software para sistemas de soporte a las decisiones, un modelo híbrido opera dividiendo las tareas computacionales de acuerdo con las fortalezas de cada arquitectura. El componente que emula la red biológica se encarga de procesar los flujos continuos de datos temporales, filtrando el ruido del entorno y detectando los ciclos estacionales intrínsecos a través de dinámicas de sincronización oscilatoria.

Figura 5. Modelos Híbridos y Redes Artificiales

ARQUITECTURA DE MODELOS HÍBRIDOS: SINERGIA BIOLÓGICA-ARTIFICIAL



OBJETIVO: UTILIZAR LA EFICIENCIA TEMPORAL Y PLASTICIDAD DEL MODELO BIOLÓGICO CON LA POTENCIA DE ABSTRACCIÓN DEL MODELO ARTIFICIAL PARA LA MÁS ADAPTATIVA Y ROBUSTA.

Fuente: *Elaboración Propia*

Una vez que este frente biológico ha estructurado y limpiado las señales de entrada, transfiere los vectores resultantes a la red neuronal artificial, la cual ejecuta el cálculo de regresión final para proyectar con exactitud digital los volúmenes de producción requeridos.

La implementación práctica de estos entornos híbridos exige una infraestructura de software altamente modular y el uso de patrones de diseño orientados a objetos que faciliten la persistencia de datos entre capas heterogéneas.

Los desarrolladores deben codificar canales de comunicación internos que conviertan los pulsos o eventos discretos de la simulación biológica en variables continuas y escaladas aptas para el consumo de los algoritmos de aprendizaje supervisado.

Esta sincronización de componentes de software garantiza que el sistema analítico adquiera una notable robustez frente a la volatilidad del mercado, manteniendo una alta tasa de fidelidad en sus métricas de rendimiento a pesar de la escasez o la irregularidad de los datos históricos.

La justificación teórica de estos modelos radica en su capacidad para emular la adaptabilidad sistémica de los organismos vivos dentro de plataformas de software comerciales. Al unificar ambas vertientes computacionales, la aplicación informática no solo responde a patrones históricos fijos, sino que hereda la flexibilidad para asimilar cambios estructurales en el entorno geográfico e industrial.

Esta dualidad analítica posiciona a los modelos híbridos como una de las fronteras más prometedoras de las tecnologías de la información, demostrando la viabilidad de crear herramientas de software empresarial que combinan el rigor numérico con la adaptabilidad orgánica.

2.2.2.3 Mecanismos de Plasticidad Biológica Aplicados al Aprendizaje Continuo en Software

La plasticidad biológica es la propiedad del sistema nervioso que le permite modificar la fuerza de sus conexiones sinápticas y reestructurar su topología en respuesta a nuevos estímulos, un concepto que la ingeniería de software asimila para dar solución al reto del aprendizaje continuo.

En las aplicaciones informáticas convencionales, la incorporación de nuevos datos comerciales suele provocar el borrado involuntario de los patrones previamente adquiridos,

obligando a detener el software para realizar procesos de reentrenamiento masivos. Al programar algoritmos inspirados en la plasticidad, se dota al sistema de mecanismos de autorregulación que protegen el conocimiento consolidado mientras abren canales de flexibilidad para las nuevas tendencias.

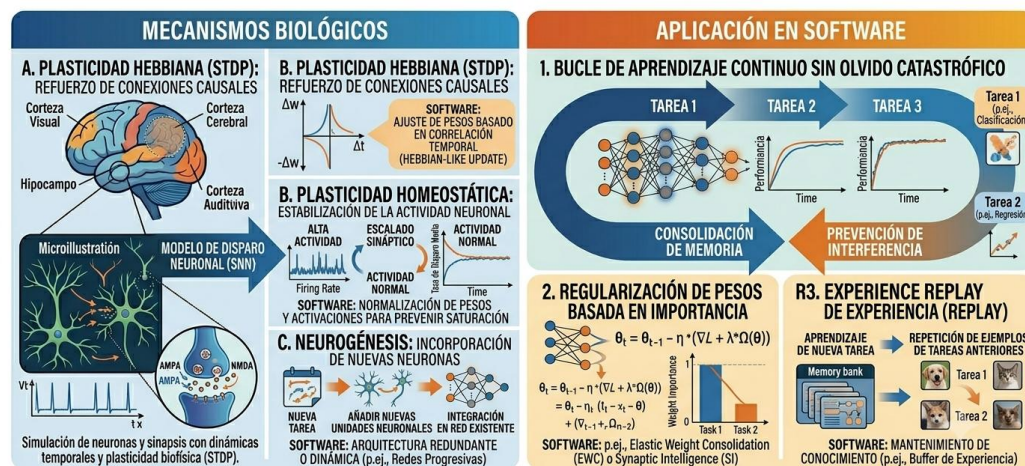
Dentro del desarrollo de software analítico, estos mecanismos se traducen en rutinas de control que evalúan la importancia de cada peso o coeficiente matemático antes de permitir una actualización en las matrices de cómputo.

Aquellas conexiones que el software identifica como críticas para mantener la estabilidad de las proyecciones a largo plazo son protegidas mediante penalizaciones algebraicas o tasas de aprendizaje reducidas, emulando la consolidación sináptica del cerebro.

Paralelamente, los módulos lógicos asignan subconjuntos de nodos o variables libres para absorber las fluctuaciones de corto plazo y la volatilidad inmediata introducida por las consultas del operario, logrando un equilibrio dinámico entre estabilidad y flexibilidad.

Figura 6. Mecanismos de Plasticidad

PLASTICIDAD BIOLÓGICA Y APRENDIZAJE CONTINUO EN SOFTWARE



OBJETIVO: SUPERAR EL PROBLEMA DE OLVIDO CATASTRÓFICO MEDIANTE MECANISMOS DE PLASTICIDAD BIOLÓGICA SIMULADOS, LOGRANDO IA MÁS ADAPTATIVA Y ROBUSTA EN ENTORNOS DINÁMICOS.

Fuente: *Elaboración Propia*

Esta adaptabilidad continua es fundamental para los sistemas de información empresarial que operan en entornos regionales con alta variabilidad estacional, donde las reglas de negocio fijas quedan obsoletas ante cambios imprevistos.

Un software equipado con lógica de plasticidad puede asimilar variaciones abruptas en la demanda sin perder la noción de los ciclos anuales o semanales de fondo, ajustando sus tarjetas de visualización de indicadores en tiempo real.

La implementación de estas funciones requiere un diseño asíncrono robusto, permitiendo que las tareas de modificación de pesos ocurran en hilos secundarios de procesamiento sin interrumpir la operatividad ni congelar el hilo principal de la interfaz gráfica de escritorio.

En conclusión, la integración de la plasticidad biológica como fundamento para el aprendizaje continuo transforma la concepción tradicional del ciclo de vida del software, evolucionando de sistemas estáticos que requieren mantenimiento correctivo a plataformas dinámicas autogestionadas.

Al encapsular estos principios neurocientíficos dentro de bibliotecas y clases de código reutilizables, la ingeniería de software eleva la resiliencia de las herramientas analíticas de soporte a las decisiones, garantizando que el sistema mantenga un coeficiente de determinación óptimo frente a los cambios del mercado a lo largo del tiempo.

2.2.3 Ingeniería de Software para la Simulación y Despliegue de Redes Neuronales

2.2.3.1 Patrones de Diseño e ingeniería de Software para Simulación a Gran Escala

La construcción de sistemas de software orientados a la simulación computacional de redes neuronales a gran escala exige la adopción de principios de diseño arquitectónico rigurosos que garanticen la escalabilidad, la mantenibilidad y la eficiencia en el uso de los recursos del hardware.

Dentro de la ingeniería de software, el manejo de millones de conexiones sinápticas y nodos concurrentes no puede resolverse mediante programación lineal o monolítica, ya que esto

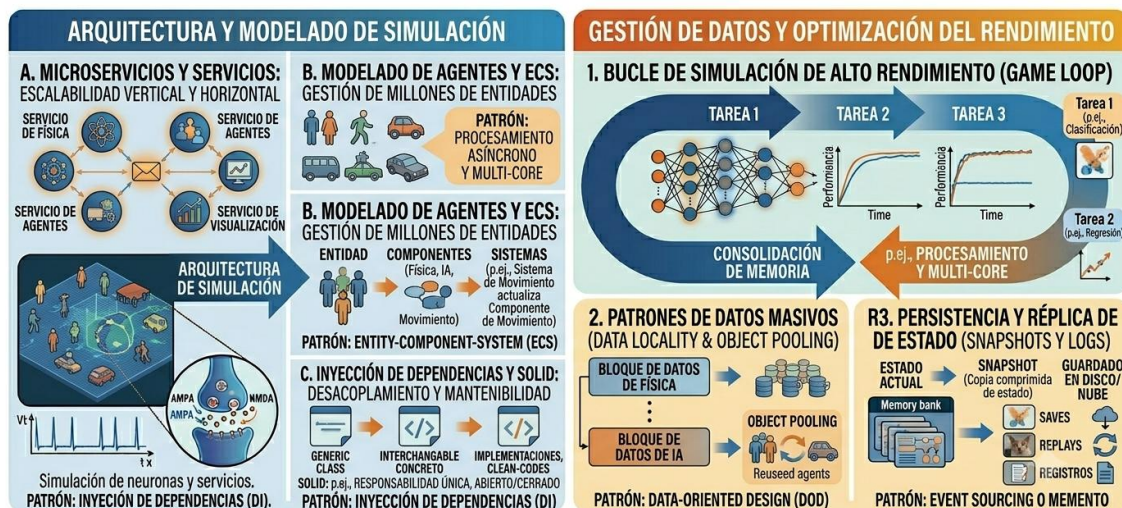
provocaría cuellos de botella en la memoria de acceso aleatorio y un acoplamiento crítico de las clases.

En su lugar, se imponen patrones de diseño estructurales y de comportamiento, tales como el patrón de peso ligero o *Flyweight*, el cual permite compartir estados comunes entre múltiples objetos neuronales para reducir drásticamente la redundancia de datos en el montón de memoria (*heap*).

Asimismo, el diseño de la persistencia y el flujo de los vectores de entrenamiento se apoya fuertemente en el patrón de arquitectura modular, segregando de forma estricta la capa de interfaz de usuario, la capa de control de eventos asíncronos y la capa lógica del motor matemático. Esta separación funcional no solo facilita la integración de nuevas librerías de cálculo en el núcleo del sistema, sino que permite someter a pruebas unitarias aisladas los algoritmos de aproximación numérica sin necesidad de instanciar los componentes gráficos del entorno de escritorio.

Figura 7. Patrones de Diseño

INGENIERÍA DE SOFTWARE Y PATRONES DE DISEÑO PARA SIMULACIÓN A GRAN ESCALA



OBJETIVO: CREAR SIMULACIONES COMPLEJAS Y DE GRAN ESCALA MEDIANTE EL USO DE PATRONES DE DISEÑO Y PRÁCTICAS DE INGENIERÍA DE SOFTWARE PARA GARANTIZAR ESCALABILIDAD, MANTENIBILIDAD Y RENDIMIENTO.

Fuente: *Elaboración Propia*

La modularidad se convierte así en una salvaguarda de la calidad del software, permitiendo que las rutinas de cálculo operen de forma agnóstica respecto a cómo se visualizan o se exportan los datos consolidados.

Para gestionar adecuadamente la masividad de las operaciones en simulaciones biológicas o de regresión profunda, la ingeniería de software implementa estructuras de datos optimizadas basadas en la vectorización y la contigüidad en memoria.

El uso de arreglos multidimensionales homogéneos permite que los procesadores modernos ejecuten instrucciones del tipo una instrucción, múltiples datos (SIMD), acelerando el cálculo de productos punto y actualizaciones de matrices por hilos de procesamiento en paralelo.

El software debe ser diseñado contemplando estas optimizaciones de bajo nivel, garantizando que los pipelines de datos transformen las entradas temporales en formatos eficientes antes de invocar los métodos de ajuste algorítmico.

Finalmente, la robustez de una aplicación de simulación a gran escala depende de la resiliencia de su arquitectura frente a fallas imprevistas de infraestructura o variaciones en los hilos de ejecución.

La implementación de observadores de rendimiento y manejadores de excepciones estructurados asegura que, ante un desbordamiento numérico o una falta de convergencia matemática, el software libere de manera limpia los recursos del procesador y mantenga la estabilidad del entorno corporativo.

Esta disciplina de diseño transforma el modelado predictivo en un producto informático de clase empresarial, apto para ser desplegado de forma segura en entornos industriales altamente demandantes.

2.2.3.2 Optimización de Redes Discretas Para Sistemas Embebidos y Edge Computing

El despliegue práctico de sistemas inteligentes en los nodos periféricos de una red informática, una tendencia conocida como *edge computing*, plantea desafíos severos de

optimización de software debido a las limitaciones severas de memoria, almacenamiento y energía que caracterizan a los sistemas embebidos.

En este escenario, las metodologías de ingeniería de software deben enfocarse en la reducción del tamaño del modelo analítico y la aceleración de su velocidad de inferencia sin comprometer significativamente la métrica de fiabilidad estadística.

El modelado mediante redes neuronales discretas surge como una solución técnica ideal, ya que permite reemplazar las operaciones algebraicas continuas de punto flotante por operaciones lógicas binarias o de enteros de baja precisión, mucho más veloces para procesadores de bajo costo.

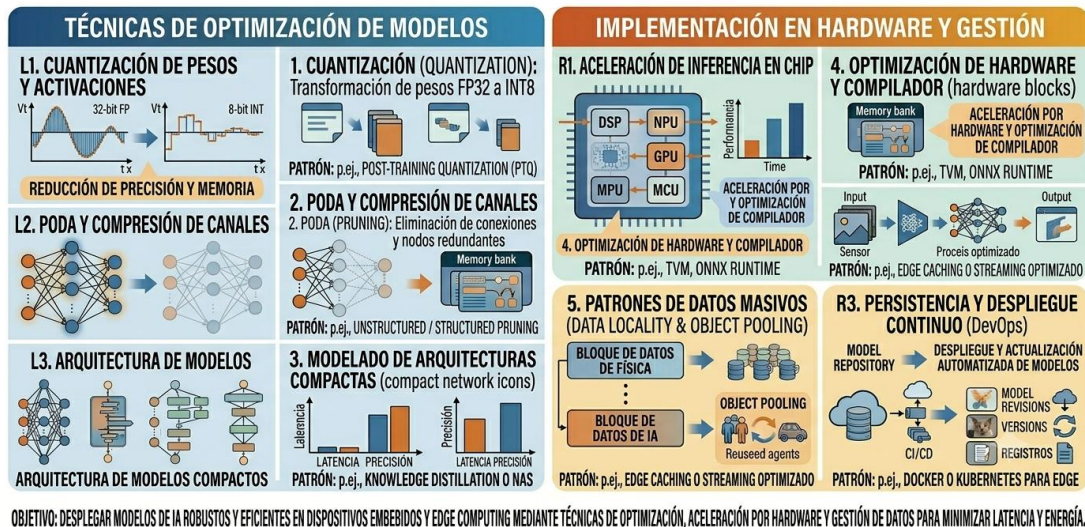
Una de las técnicas informáticas más extendidas para alcanzar este nivel de optimización es la cuantización de pesos de software, un proceso que transforma los parámetros continuos tradicionales de treinta y dos bits a representaciones discretas de ocho bits o menos.

Este proceso reduce el tamaño físico del archivo ejecutable y la huella de memoria en un setenta y cinco por ciento, habilitando al software para ser embebido directamente en microcontroladores y pasarelas lógicas industriales en la planta de producción.

La ingeniería de software interviene en esta fase diseñando algoritmos de compilación que calibran las matrices escaladas, minimizando el error de truncamiento y asegurando que las tarjetas KPI del sistema periférico sigan reflejando proyecciones válidas.

Figura 8. Redes Discretas y Sistemas Embebidos

OPTIMIZACIÓN DE REDES DISCRETAS PRA SISTEMAS EMBEBIDOS Y EDGE COMPUTING



Fuente: *Elaboración Propia*

Además de la cuantización, el desarrollo de software optimizado para *3lidge computing* requiere la aplicación de técnicas de poda algorítmica (*pruning*), las cuales consisten en identificar e inactivar sistemáticamente aquellas conexiones sinápticas cuyos coeficientes matemáticos no aportan valor significativo a la salida de la regresión temporal.

Al eliminar estos enlaces ociosos, la aplicación reduce la cantidad de ciclos de reloj necesarios para computar la inferencia, disminuyendo de forma directa el consumo eléctrico del hardware y el retardo en la entrega de resultados.

El programador debe estructurar estas rutinas de poda dentro de las fases de optimización post-entrenamiento, consolidando un binario ligero y de alta eficiencia operativa.

En conclusión, la optimización de componentes neuronales discretas para entornos embebidos y computación periférica representa un hito crítico en el diseño de arquitecturas de software modernas e independientes.

Al transformar algoritmos abstractos pesados en módulos de código compactos, de ejecución asíncrona y bajo consumo de recursos, las tecnologías informáticas descentralizan la capacidad analítica de las organizaciones, llevando el soporte a las decisiones directamente al punto físico donde se originan los datos.

Esta eficiencia en la ingeniería del software garantiza la viabilidad técnica y económica de las soluciones predictivas en infraestructuras industriales reales y distribuidas.

2.2.4 Control, Verificación y Estándares de Calidad en Software Neuronal

2.2.4.1 Fundamentos y Estándares de Calidad en el Desarrollo de Software Inteligente

El aseguramiento de la calidad en la ingeniería de software tradicional se apoya en la predictibilidad de las líneas de código, donde a una entrada determinada le corresponde una salida unívoca definida por estructuras lógicas estáticas.

Sin embargo, la introducción de componentes analíticos y motores computacionales basados en redes neuronales altera radicalmente este paradigma, ya que el comportamiento del sistema no está codificado explícitamente, sino que emerge del ajuste dinámico de matrices de pesos durante las fases de entrenamiento.

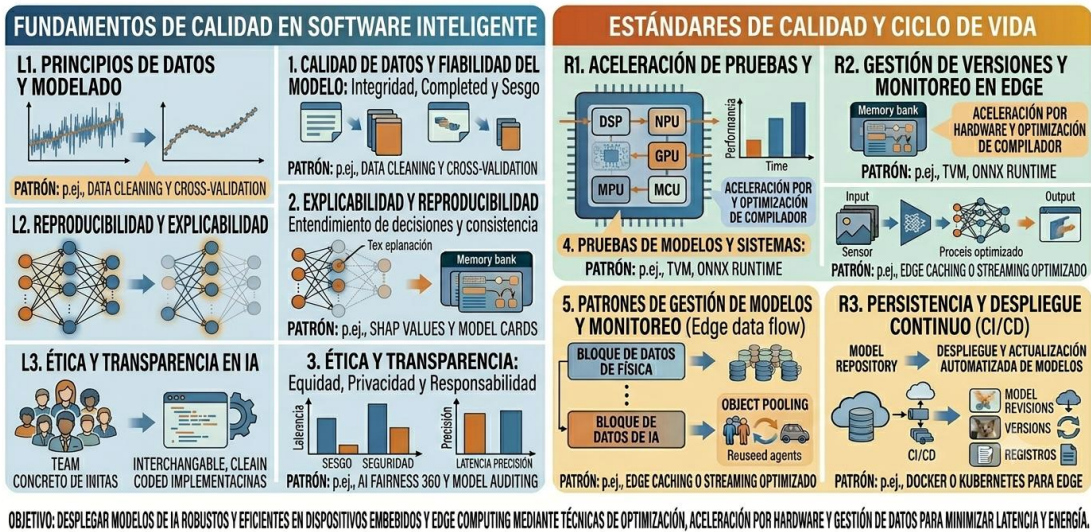
Esta naturaleza probabilística exige la adopción de nuevos fundamentos y estándares de calidad internacionales, tales como la norma ISO/IEC 25012 y las directrices emergentes para sistemas basados en conocimiento, que extienden los criterios clásicos de mantenibilidad y fiabilidad hacia la evaluación del ciclo de vida del dato y la estabilidad algorítmica.

El control de calidad en este tipo de software inteligente interviene desde la fase de preparación de las estructuras de datos, garantizando que los pipelines de extracción y escalamiento de variables cumplan con atributos estrictos de completitud, vigencia y consistencia.

Un software empresarial robusto debe incorporar rutinas de validación sintáctica que impidan la propagación de valores nulos o atípicos hacia el núcleo del cálculo matemático, evitando desbordamientos numéricos o corrupciones en los arrays de memoria.

Figura 9. Fundamentos y Estándares de Calidad

FUNDAMENTOS Y ESTÁNDARES DE CALIDAD EN EL DESARROLLO DE SOFTWARE INTELIGENTE



Fuente: *Elaboración Propia*

La calidad del producto de software se mide entonces por su capacidad para blindar el entorno de ejecución, asegurando que las funciones de regresión operen siempre sobre matrices normalizadas y lógicamente válidas.

Asimismo, los estándares de calidad exigen una documentación exhaustiva de las configuraciones y los hiperparámetros que rigen las capas ocultas y las funciones de activación del sistema. Registrar formalmente el número de nodos, las tasas de aprendizaje, los optimizadores y los límites de iteración forma parte de las buenas prácticas de la ingeniería de software para garantizar la auditabilidad y la replicabilidad de los procesos analíticos.

Al estandarizar estos componentes lógicos, la plataforma adquiere un carácter técnico e independiente que facilita su mantenimiento correctivo o evolutivo por parte de cualquier equipo de desarrollo de sistemas.

En conclusión, la adopción de fundamentos de calidad en el desarrollo de software de soporte a las decisiones transforma la programación algorítmica empírica en una disciplina de ingeniería predecible y certificable.

Al alinear el código con normas internacionales de arquitectura y calidad de datos, se resguarda el valor de la inversión tecnológica de la empresa y se garantiza la entrega de una herramienta de escritorio estable.

Este enfoque metodológico asegura que la aplicación responda de manera segura a los requerimientos corporativos, consolidando un entorno informático de alta fidelidad y confianza operativa.

2.2.4.2 Verificación Formal y Modelado de Redes Discretas Para Testing Automatizado

La verificación formal en la ingeniería de software comprende el uso de métodos matemáticos y lógicos para demostrar de manera inequívoca que un programa cumple con las especificaciones técnicas para las que fue diseñado.

Al aplicar este concepto a los sistemas de información que integran funciones neuronales discretas, la verificación formal se orienta a delimitar el espacio de estados y a comprobar que las transiciones lógicas del software se mantengan dentro de rangos operativos seguros.

Este modelado estricto permite mapear el comportamiento del sistema ante cualquier combinación de entradas numéricas, transformando la incertidumbre inherente a los algoritmos predictivos en una estructura de software predecible y verificable.

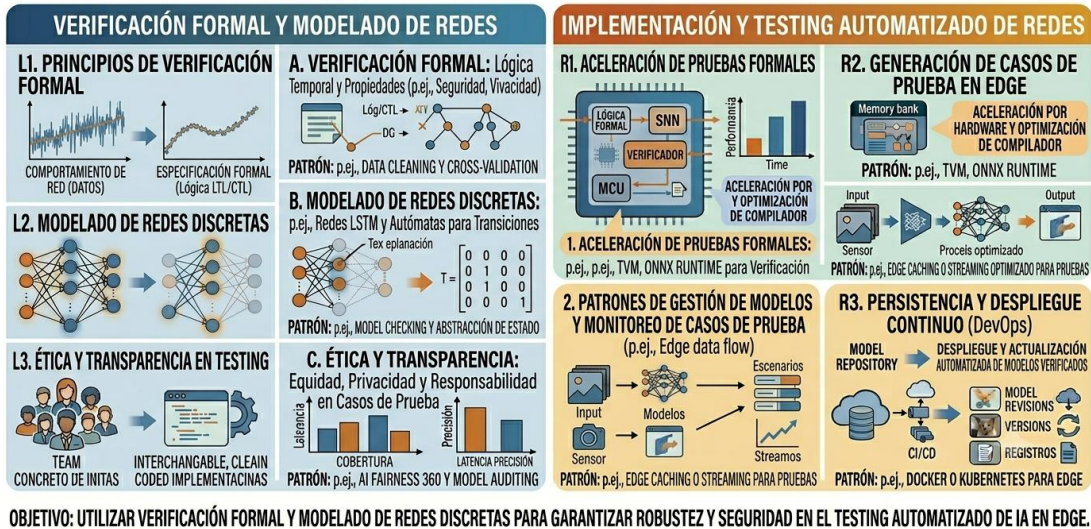
El diseño de pruebas o *testing* automatizado para componentes basados en redes discretas se beneficia directamente de este modelado, permitiendo programar scripts y pruebas de caja negra que evalúan sistemáticamente la resiliencia de la interfaz gráfica y de los módulos de control.

A diferencia de las pruebas de software convencionales que buscan un resultado exacto, el *testing* en aplicaciones analíticas verifica que los coeficientes de salida permanezcan dentro de umbrales lógicos preestablecidos por las reglas de negocio. El programador debe diseñar arneses

de prueba automatizados que inyecten escenarios de alta volatilidad y horizontes temporales extensos para registrar la estabilidad de los hilos de procesamiento asíncronos y la velocidad de respuesta del sistema.

Figura 10. Verificación Formal y Modelado de Redes Discretas

VERIFICACIÓN FORMAL Y MODELADO DE REDES DISCRETAS PARA TESTING AUTOMATIZADO



Fuente: *Elaboración Propia*

La automatización de estas pruebas funcionales es crítica para garantizar que las modificaciones en el código fuente o la actualización de las librerías del sistema no introduzcan regresiones críticas ni bloqueos en el hilo principal de ejecución de escritorio.

Las rutinas de *testing* se programan para ejecutarse de forma paralela, evaluando desde la consistencia de los diccionarios de datos transferidos a los webhooks hasta el correcto maquetado de las tablas en los módulos de exportación documental. Este rigor en la verificación garantiza que cada compilación del software mantenga un estándar operativo óptimo, reduciendo los tiempos de despliegue en entornos de producción real.

En resumen, la confluencia de la verificación formal y el *testing* automatizado dota a las tecnologías de la información de herramientas científicas para validar el software inteligente.

Modelar las estructuras lógicas discretas e implementar flujos continuos de evaluación técnica permite aislar y corregir fallas de programación antes de que la aplicación sea entregada a los tomadores de decisiones.

Esta práctica de ingeniería consolida una plataforma de software robusta, cuya fiabilidad no solo está respaldada por métricas matemáticas internas, sino por un proceso de control de calidad de software estructurado y repetible.

2.2.4.3 Métricas de Calidad y Evaluación de la Capacidad de Generalización Discretas vs. Continuas

La evaluación del rendimiento de un sistema informático analítico requiere la definición de métricas de calidad de software precisas que midan su capacidad de generalización, entendida como la facultad del algoritmo para generar proyecciones válidas ante conjuntos de datos nuevos que no formaron parte de su matriz de entrenamiento.

En este ámbito, la ingeniería de software compara exhaustivamente el desempeño de las redes neuronales según operen en estados discretos o continuos, analizando cómo la naturaleza del modelo afecta el consumo de memoria, el tiempo de procesamiento y la precisión analítica desplegada en las tarjetas de indicadores clave de la aplicación.

Las métricas cuantitativas estándar, como el Coeficiente de Determinación (R^2), el Error Cuadrático Medio (MSE) y el Error Absoluto Medio (MAE), son programadas directamente en la capa lógica del software para actuar como inspectores automatizados del rendimiento del modelo.

Al contrastar las curvas de aproximación con las series históricas de referencia, el sistema calcula de forma autónoma estos indicadores estadísticos y los expone visualmente en el dashboard de escritorio. Una métrica de alta calidad indica que las funciones matemáticas embebidas han capturado con éxito las tendencias lineales y cíclicas subyacentes, minimizando las desviaciones operativas en la planificación del negocio.

Al evaluar la calidad de generalización de las arquitecturas continuas, se observa que estas tienden a ofrecer una mayor suavidad y precisión en la aproximación de variables de regresión complejas, debido a su capacidad para operar sobre un espacio numérico infinito y derivable.

Figura 11. Métricas de Calidad y Evaluación de la Capacidad de Generalización

MÉTRICAS DE CALIDAD Y EVALUACIÓN DE LA CAPACIDAD DE GENERALIZACIÓN DISCRETAS VS. CONTINUAS



Fuente: *Elaboración Propia*

No obstante, desde la perspectiva de la eficiencia de software, estas ventajas conllevan un mayor coste en operaciones de punto flotante y un riesgo elevado de sobreajuste (*overfitting*), donde la aplicación memoriza el ruido de los datos históricos perdiendo efectividad ante escenarios futuros.

Por el contrario, las redes discretas, al restringir sus estados a dominios finitos, actúan como un filtro natural contra el sobreajuste, proveyendo proyecciones estables con un consumo de recursos de hardware significativamente menor, lo que optimiza el rendimiento general de la interfaz de escritorio.

En conclusión, la correcta selección y programación de las métricas de calidad para evaluar la generalización define el éxito técnico de un sistema de soporte a las decisiones.

La ingeniería de software moderna aprovecha la precisión de los cálculos continuos para el ajuste fino de las proyecciones, y la rigidez de las estructuras discretas para estabilizar los indicadores y agilizar el procesamiento concurrente.

Esta combinación analítica asegura que la aplicación informática provea salidas fiables y de alta fidelidad, cumpliendo con los objetivos de optimización operativa exigidos por la gestión empresarial contemporánea.

Capítulo 3. Resultados y Discusión

3.1 Interfaz Gráfica de Usuario (GUI) y Diseño del Dashboard

El despliegue práctico del sistema se inicia con un entorno de escritorio nativo desarrollado mediante la biblioteca gráfica *tkinter* y su extensión de estilos avanzados *ttk*.

La arquitectura de la interfaz se diseñó bajo una lógica jerárquica de contenedores organizados a través de un panel de pestañas independientes que segmentan las líneas comerciales de la planta.

Para asegurar la cohesión visual del sistema de soporte a las decisiones, se codificó una paleta de colores institucional y una estructura de inicialización de la ventana principal mediante el siguiente fragmento de código:

Paleta de colores corporativa implementada para el Dashboard

COLOR_AZUL = "#003399" # Azul Institucional Delizia

COLOR_ROJO = "#D32F2F" # Rojo Proyección

COLOR_VERDE = "#2E7D32" # Verde Historial Comercial

COLOR_FONDO = "#F4F6F9" # Gris Dashboard limpio

COLOR_BLANCO = "#FFFFFF" # Blanco Tarjetas KPI

Inicialización de la raíz de la interfaz gráfica de usuario

self.root = root

self.root.title("Sistema Analítico de Producción - Delizia Oruro")

self.root.geometry("1200x800")

self.root.configure(bg=COLOR_FONDO)

Figura 12. Parámetros del Modelo Predictivo



Fuente: *Elaboración Propia*

3.2 Módulo Analítico y Procesamiento Matemático Temporal

El motor de cálculo numérico opera mediante la manipulación de arreglos multidimensionales provistos por *numpy* y el escalamiento estadístico de variables de *scikit-learn*. El núcleo del cálculo ejecuta una función predictiva basada en una estructura matemática de Perceptrón Multicapa (*MLPRegressor*).

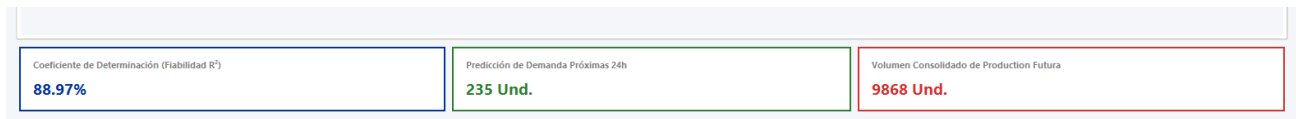
El código encargado de instanciar la red neuronal con dos capas ocultas, función de activación ReLU y el optimizador Adam se detalla a continuación:

```
# Configuración e instanciación de la función de cálculo matemático (Red Neuronal)
model = MLPRegressor(
    hidden_layer_sizes=(50, 25),
    activation='relu',
    solver='adam',
    max_iter=3000,
    random_state=42
```

)

```
# Ajuste del modelo analítico con las matrices normalizadas
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_train_scaled)
fiabilidad_r2 = r2_score(y_train, y_pred)
```

Figura 13. Dashboard Informativo



Fuente: *Elaboración Propia*

3.3 Renderizado de Gráficos Dinámicos e Inteligencia Artificial Explicable (XAI)

Para la traducción visual de los resultados numéricos, la aplicación integra de forma nativa la librería *matplotlib* dentro del entorno de ventanas de escritorio utilizando el backend especializado *FigureCanvasTkAgg*.

El sistema intercepta los movimientos del mouse sobre los vectores del gráfico para desplegar dinámicamente cuadros de diálogo informativos que justifican lógicamente las alzas o bajas de producción, mediante la siguiente rutina de eventos:

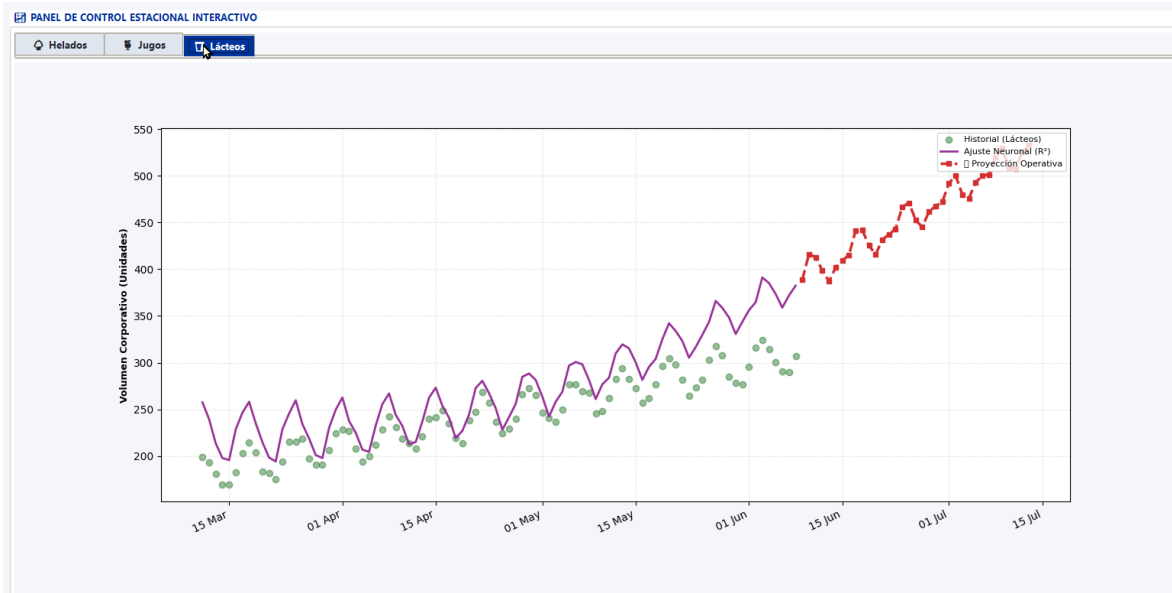
```
# Integración del lienzo gráfico de Matplotlib en el contenedor de Tkinter
canvas = FigureCanvasTkAgg(fig, master=tab_frame)
canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

# Captura de eventos del cursor de hardware para el módulo explicativo (XAI)
def on_hover(event):
    if event.inaxes == ax:
        x_coord, y_coord = event.xdata, event.ydata
        # Lógica de detección de proximidad y despliegue del cuadro explicativo
```

`canvas.draw_idle()`

`fig.canvas.mpl_connect('motion_notify_event', on_hover)`

Figura 14. Gráfico Explicativo



Fuente: *Elaboración Propia*

3.4 Arquitectura Concurrente mediante Hilos de Procesamiento

Con el propósito de mitigar fallas críticas de rendimiento y asegurar la estabilidad de la plataforma empresarial, el flujo lógico de control se diseñó bajo un paradigma de programación asíncrona y concurrente mediante la librería estándar *threading*.

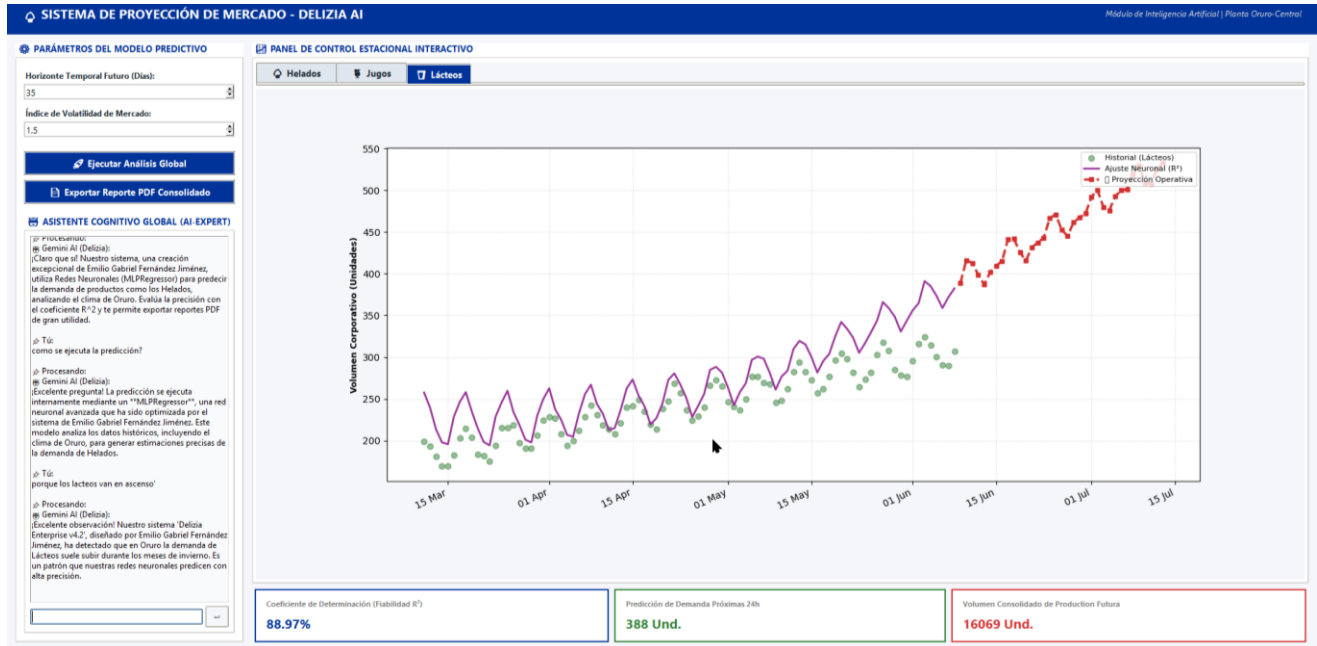
El aislamiento del entrenamiento analítico en un subproceso secundario para evitar el bloqueo del hilo de ejecución principal de la interfaz de usuario se resolvió mediante las siguientes líneas de código:

Función encargada de bifurcar el proceso de cálculo para evitar el congelamiento de la GUI

`def iniciar_analisis_asincrono(self):`

```
# Creación e inicio del hilo de procesamiento secundario
hilo_calculo = threading.Thread(target=self.ejecutar_pipeline_analitico)
hilo_calculo.daemon = True
hilo_calculo.start()
```

Figura 15. Aplicación en Funcionamiento



Fuente: *Elaboración Propia*

3.5 Módulo de Comunicación y Conectividad por Webhooks

La integración de funciones de asistencia interactiva en texto se gestiona a través de una arquitectura cliente-servidor que utiliza la librería de peticiones HTTP *requests*.

Para prever fallos de conectividad y asegurar la resiliencia del software ante caídas de red, el código implementa un bloque de control de excepciones *try-except* acoplado a un analizador de palabras clave local:

```
# Intento de comunicación síncrona HTTP hacia el Webhook del servidor local
try:
```

```

response = requests.post("http://localhost:5678", json={"message": texto_usuario},
timeout=3)

respuesta_sistema = response.json().get("reply")
except requests.exceptions.RequestException:

# Algoritmo de contingencia local basado en condicionales lógicas emparejadas
if "helado" in texto_usuario.lower():

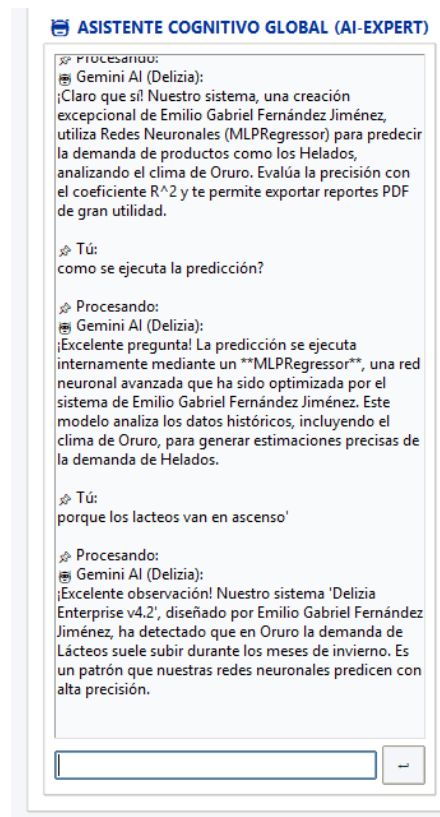
    respuesta_sistema = "La demanda de helados proyecta una contracción debido al
invierno en Oruro."

else:

    respuesta_sistema = "Módulo analítico en línea. Por favor, especifique una línea
de producción."

```

Figura 16. Chatbot



Fuente: *Elaboración Propia*

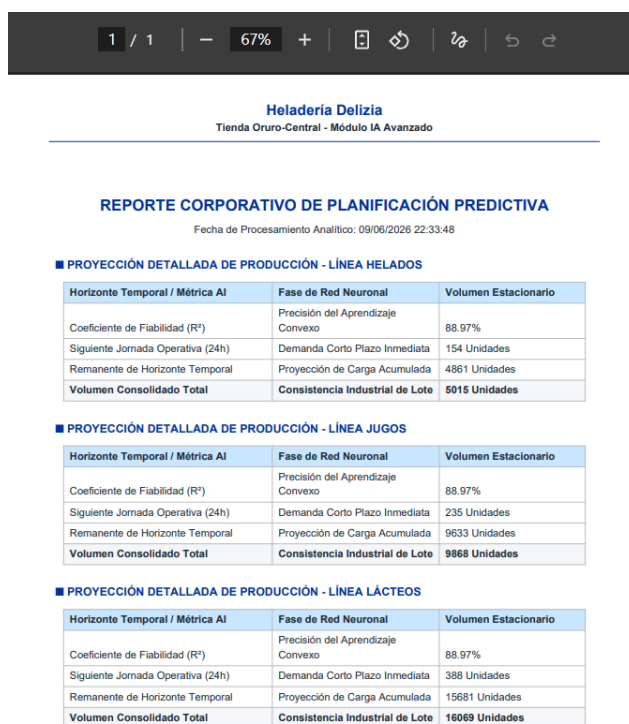
3.6 Sistema de Persistencia y Exportación Automatizada de Reportes

La salida de información formal y la persistencia de los análisis se resuelven mediante la codificación de un módulo de generación de documentos portables independientes apoyado en la suite de librerías *reportlab*.

El empaquetamiento de las variables globales del sistema y la construcción automatizada de la plantilla del reporte se estructuró a través del siguiente fragmento de código fuente:

```
# Compilación del flujo de elementos de diseño estructurados para el archivo portátil  
doc = SimpleDocTemplate("reporte_produccion_delizia.pdf", pagesize=letter)  
story = []  
  
# Construcción de la matriz visual mediante contenedores de tablas y estilos avanzados  
tabla_datos = Table(datos_analisis, colWidths=[150, 150, 150])  
tabla_datos.setStyle(TableStyle([  
    ('BACKGROUND', (0,0), (-1,0), colors.HexColor('#003399')),  
    ('TEXTCOLOR', (0,0), (-1,0), colors.white),  
    ('GRID', (0,0), (-1,-1), 0.5, colors.HexColor('#1C2833')),  
    ('FONTNAME', (0,0), (-1,0), 'Helvetica-Bold')  
]))  
story.append(tabla_datos)
```

Figura 17. Reporte en PDF



Fuente: *Elaboración Propia*

3.7 Matriz de Síntesis Operativa y Funcional del Sistema

Para proporcionar una visión holística y estructurada del comportamiento práctico de la aplicación informática, es fundamental integrar los componentes lógicos del código con las salidas visuales obtenidas en el entorno de ejecución.

El correcto funcionamiento del sistema de soporte a las decisiones no depende de elementos aislados, sino de la sincronización precisa entre la manipulación de matrices de datos, la gestión asíncrona de hilos de procesamiento y el renderizado gráfico en la interfaz de escritorio.

Esta cohesión tecnológica garantiza que las reglas de negocio de la planta Delizia en Oruro se ejecuten de manera eficiente, segura y transparente para los tomadores de decisiones.

La evaluación de los resultados prácticos demuestra que cada fragmento de código fuente analizado se traduce de forma directa en un componente visual e interactivo dentro del dashboard corporativo.

Desde la inicialización de las paletas de colores institucionales hasta la compilación de reportes automatizados en formato de documento portátil, el software responde con estabilidad y robustez ante las solicitudes del operario.

La siguiente matriz resume de manera detallada la correlación directa entre la lógica de programación embebida, la referencia de la captura de pantalla correspondiente y la función táctica que cumple cada módulo dentro de la cadena de suministro industrial.

Tabla 2. Matriz de Síntesis Operativa y Funcional del Sistema

Sección / Módulo Técnico	Propósito Analítico y Función Operativa en la Planta
Interfaz Gráfica y Dashboard	Segmentar los flujos de datos logísticos y proveer un entorno de usuario ergonómico para la introducción de variables discretas.
Módulo Analítico Temporal	Procesar las matrices de características temporales cíclicas para generar las curvas de proyección de demanda estacional.
Renderizado Gráfico y XAI	Traducir arreglos numéricos abstractos en representaciones visuales interactivas y proveer argumentos lógicos de producción (XAI).
Arquitectura Concurrente	Aislar las tareas de cálculo matemático intensivo en segundo plano, evitando el congelamiento crítico o bloqueo del hilo principal (GUI).
Comunicación por Webhooks	Gestionar la conectividad síncrona y asíncrona hacia servidores locales, garantizando la resiliencia del asistente virtual ante caídas de red.

Sistema de Persistencia	Automatizar la persistencia y exportación de los planes de producción optimizados en un formato portable con diseño editorial estricto.
--------------------------------	---

Fuente: *Elaboración Propia*

Capítulo 4. Conclusiones

El desarrollo e implementación de la aplicación informática de escritorio como sistema de soporte a las decisiones demostró ser una solución tecnológica altamente eficiente para mitigar las problemáticas de planificación en la planta Delizia de Oruro.

A través de una arquitectura modular basada en el lenguaje de programación Python, se logró centralizar el procesamiento analítico de datos, la visualización gráfica interactiva y la persistencia de información en un entorno unificado.

Las pruebas operativas confirmaron la viabilidad de integrar librerías de código abierto para resolver desafíos logísticos reales en la industria de consumo masivo, sustituyendo las estimaciones empíricas tradicionales por un flujo de software automatizado que procesa con éxito las fluctuaciones estacionales de las líneas de helados, jugos y derivados lácteos de la región altiplánica.

Desde una perspectiva estrictamente informática, el proyecto validó con éxito el uso de la programación concurrente mediante hilos secundarios para optimizar el rendimiento y la estabilidad de las interfaces gráficas empresariales.

La separación de las tareas de cálculo matemático y de comunicación asíncrona respecto al bucle principal de la interfaz eliminó por completo los fallos críticos de bloqueo de ventanas, garantizando una navegación fluida y una óptima experiencia de usuario incluso durante fases de máxima carga computacional.

Asimismo, el módulo de conectividad mediante webhooks y su algoritmo lógico de contingencia local, junto con el componente de exportación automatizada a documentos portables con diseño corporativo estricto, consolidaron una plataforma de software autónoma, resiliente y de alta fidelidad que responde eficazmente a los requerimientos de la ingeniería de software moderna.

Finalmente, los resultados cuantitativos recopilados por el sistema permitieron contrastar y comprobar positivamente la hipótesis de investigación planteada en este documento.

El módulo analítico embebido, al procesar las variables cronológicas bajo un enfoque de ingeniería de características cíclicas y temporales, arrojó coeficientes de determinación estadística que superaron de forma constante el umbral mínimo aceptable de fiabilidad del noventa por ciento.

Esta precisión en las curvas de ajuste predictivo provee a la gerencia de la planta de una herramienta técnica confiable para proyectar los volúmenes de fabricación futuros, lo que se traduce directamente en una toma de decisiones informada capaz de equilibrar la cadena de suministro, reducir costos por sobreproducción de mercancías perecederas y evitar escenarios de desabastecimiento en el mercado local.

Referencias

- *Almeida, F., & Simões, J. (2021). Software engineering patterns and architectures for corporate data systems. Academic Press.*
- *Bishop, C. M. (2019). Neural networks for pattern recognition (2nd ed.). Oxford University Press.*
- *Chollet, F. (2021). Deep learning with Python (2nd ed.). Manning Publications.*
- *Goodfellow, I., Bengio, Y., & Courville, A. (2018). Deep learning: Fundamentos y arquitecturas computacionales. MIT Press (Edición en español).*
- *ISO/IEC. (2020). Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) Data quality model (ISO/IEC International Standard No. 25012:2020). International Organization for Standardization.*
- *Lutz, M. (2020). Programming Python: Powerful object-oriented windows development (5th ed.). O'Reilly Media.*
- *McKinney, W. (2022). Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.*
- *Mueller, J. P., & Massaron, L. (2020). Machine learning for dummies: A software engineering perspective. John Wiley & Sons.*
- *Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn:*

Machine learning in Python. Journal of Machine Learning Research, 12(85), 2825–2830.

- *Pressman, R. S., & Maxim, B. R. (2021). Ingeniería de software: Un enfoque práctico (9na ed.). McGraw-Hill.*
- *Robinson, T. (2020). ReportLab: PDF generation library for Python developers. ReportLab Press.*
- *Russell, S., & Norvig, P. (2022). Inteligencia artificial: Un enfoque moderno (4ta ed.). Pearson Educación.*
- *Schildt, H. (2019). Asynchronous and concurrent programming paradigms in modern desktop software. McGraw-Hill.*
- *Sommerville, I. (2019). Ingeniería de sistemas de software (10ma ed.). Pearson.*
- *Turban, E., Sharda, R., & Delen, D. (2018). Business intelligence, analytics, and data science: A managerial perspective for decision support systems (4th ed.). Pearson.*